# CNN và ResNet

Từ nét cục bộ đến nhãn của ảnh, từ 2 lớp đến 50 lớp

Chương 5 · Bài 5.2

# Cần một nhãn cho cả ảnh

Một ảnh có thể chứa nét ngang, nét đứng và nhiễu.

Convolution tạo feature map, còn head gộp feature thành nhãn của cả ảnh.

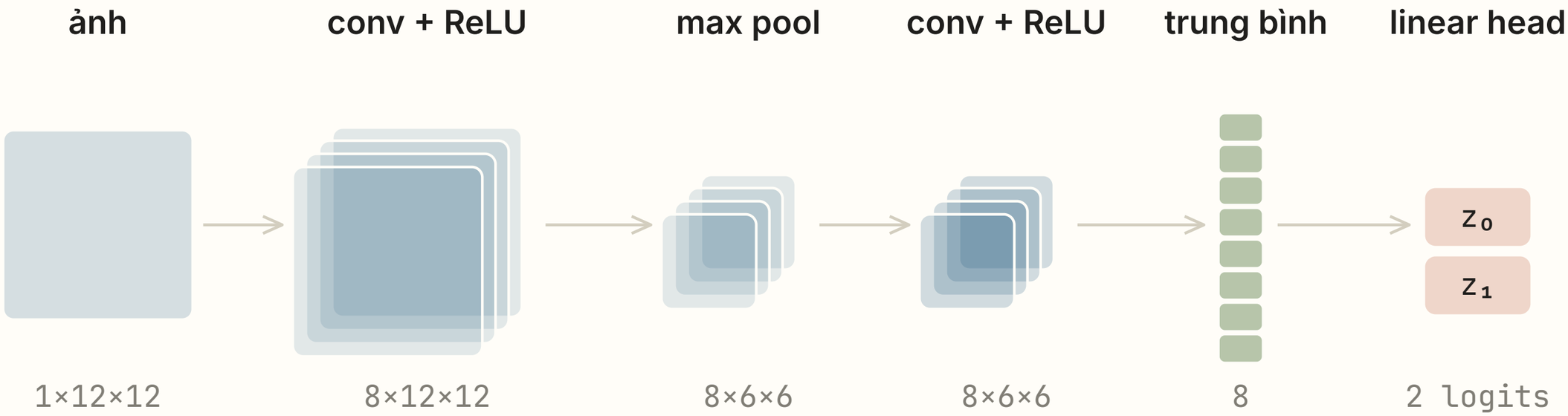
# Mục tiêu

- Theo dõi shape từ ảnh tới đầu ra của CNN.
- Huấn luyện một CNN nhỏ.
- Viết residual block; hiểu vì sao đường đi tắt giúp mạng sâu học được.
- Đọc cấu trúc ResNet và đếm tham số.

01

# Xây CNN nhỏ

# Theo dõi shape




TinyCNN: hai lần conv + ReLU xen max pool, lấy trung bình mỗi kênh, rồi linear ra hai logits.

# Activation và head

Thiếu ReLU, chồng conv vẫn chỉ là một phép affine.

Global average pooling biến mỗi kênh thành một số.

Head luôn nhận 8 số, dù ảnh lớn hay nhỏ.

Hai logits cho hai lớp; cross-entropy nhận logits trực tiếp.

HỎI NHANH

## Head trả về logits `(N, 2)`. Nhãn đưa vào `CrossEntropyLoss` cùng logits ấy phải có shape và kiểu gì?

A `(N, 2)`, one-hot kiểu `torch.long`

B `(N, 1)`, float chứa 0.0 hoặc 1.0

C `(N,)`, kiểu `torch.long`, chứa chỉ số lớp

D `(N, 1, 12, 12)`, mask nhị phân theo từng pixel

# Ảnh tự sinh để kiểm cơ chế

```
train_images, train_labels = make_lines(80, 1)
valid_images, valid_labels = make_lines(40, 2)
print(tuple(train_images.shape), torch.bincount(train_labels).tolist())
print(tuple(valid_images.shape), torch.bincount(valid_labels).tolist())
```

KẾT QUẢ

```
(80, 1, 12, 12) [40, 40]
(40, 1, 12, 12) [20, 20]
```

# Tám ảnh train đầu tiên

8 ảnh train đầu tiên của make_lines (12×12, nhiễu σ = 0.04)



đứng ngang đứng ngang đứng ngang đứng ngang

chỉ 5 vị trí × 2 hướng: validation khác train ở nhiễu

Tám ảnh train: nét đứng hoặc nét ngang, ở năm vị trí, cộng nhiễu nhỏ.

HỎI NHANH

`make_lines` chọn vị trí nét bằng `rng.integers(3, 8)`. Bỏ nhiễu đi thì có bao nhiêu ảnh sạch khác nhau, và validation kiểm được điều gì?

A 5 ảnh, validation kiểm cả vị trí lẫn hướng nét mới

B 10 ảnh, validation chỉ khác train ở phần nhiễu

C 10 ảnh, validation có những vị trí mà train chưa gặp

D 80 ảnh, vì mỗi ảnh train là một mẫu riêng

# Kiểm trước khi train

Chạy thử hai ảnh trước khi train.

```
model = TinyCNN()
print(tuple(model(train_images[:2]).shape))
print("parameters:", sum(p.numel() for p in model.parameters()))
```

KẾT QUẢ

```
(2, 2)
parameters: 682
```

# Đếm tham số

$$80 + 584 + 18 = 682 \text{ tham số.}$$

$8(1 \cdot 3^2 + 1) = 80$; $8(8 \cdot 3^2 + 1) = 584$; $2(8 + 1) = 18$. Mỗi số $+1$ là bias của một kênh ra.

HỎI NHANH

## Tensor trước head có shape `(8, 16, 4, 4)`. Gọi `flatten(0)` thay cho `flatten(1)` thì chuyện gì xảy ra?

A Cả batch bị gộp thành một vector `(2048,)`

B Chiều kênh và chiều batch đổi chỗ, ra `(16, 128)`

C Chỉ hai chiều không gian bị gộp, ra `(8, 16, 16)`

D Không đổi gì, vì PyTorch tự bỏ qua chiều batch

HỎI NHANH

Đổi conv thứ hai thành `Conv2d(8, 16, 3, padding=1)` và head thành `Linear(16, 2)`. TinyCNN mới có bao nhiêu tham số?

A 682

B 1264

C 1266

D 1282

02

# Huấn luyện và đọc kết quả

# Một bước cập nhật

Mỗi bước dùng cả 80 ảnh; Adam, cross-entropy.

```
for step in range(100):
    model.train()
    optimizer.zero_grad()
    logits = model(train_images)
    loss = criterion(logits, train_labels)
    loss.backward()
    optimizer.step()
```

# Kết quả

Mỗi cặp là `(loss, accuracy)`; loss đã làm tròn bốn chữ số.

KẾT QUẢ

```
before: (0.6981, 0.5)
train: (0.0, 1.0)
valid: (0.0, 1.0)
```

Validation từ 0.5, mức đoán bừa, lên 1.0.

Kết luận chỉ trong quy tắc sinh: 10 ảnh sạch cộng nhiễu.

HỎI NHANH

## Đọc thấy `train: (0.0, 1.0)` và `valid: (0.0, 1.0)`, một bạn kết luận “model overfit, vì loss train đã bằng 0”. Nhận xét nào đúng?

A Đúng, loss train về 0 là dấu hiệu học thuộc

B Sai, overfit cần train tốt hơn validation một cách rõ rệt

C Đúng, vì mọi xác suất dự đoán đều bằng 1

D Sai, vì loss bằng 0 nghĩa là model đúng trên mọi ảnh

# CNN đã nhìn thấy gì?

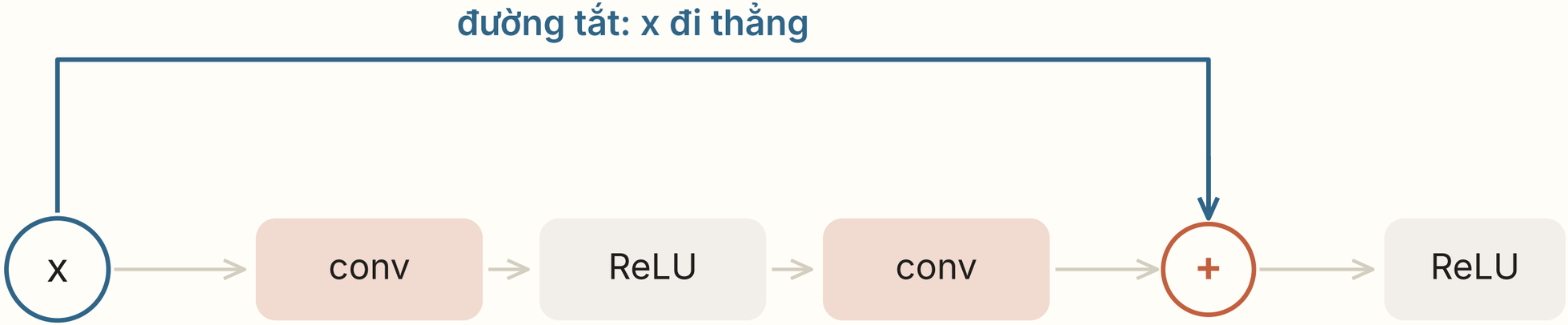



Ba kênh sau lớp conv đầu: mỗi kênh nhấn một phần khác nhau của nét.

03

# Giữ đường truyền thông tin

# Học phần cần sửa




F(x)

$$y = \mathrm{ReLU}(x + F(x))$$

Residual block: đường tắt cộng thẳng x vào F(x) rồi mới qua ReLU.

# Gradient có đường tắt

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y}\left(I + \frac{\partial F}{\partial x}\right)$$

Tích qua nhiều block luôn chứa số hạng $I$: một lối đi không qua nhánh nào.

HỎI NHANH

Mười block một chiều số $y = x + F(x)$, không có ReLU, block nào cũng có $\partial F/\partial x = a$. Gradient đi qua cả mười block được nhân với bao nhiêu khi $a = -0.5$, và khi $a = 0.5$?

A $-4$ và $6$

B Đúng bằng $1$ ở cả hai trường hợp

C Khoảng $0.001$ và khoảng $57.7$

D $0.5^{10}$ ở cả hai trường hợp

# Một block residual trên hai số

$$(2, -1) + (0.3, 0.4) = (2.3, -0.6)$$

Có ReLU sau phép cộng: $(2.3, 0)$. Phải nói rõ ReLU đặt ở đâu.

# Kiểm nhánh bằng 0

KẾT QUẢ

```
True
[[2.0, 0.0]]
```

$F = 0$: dữ liệu dương đi qua nguyên vẹn, nhưng ReLU sau phép cộng vẫn cắt $x = (2, -1)$ thành $(2, 0)$.

Đổi kênh hoặc stride thì đường tắt cần projection để khớp shape.

HỎI NHANH

Nhánh chính cho ra `(N, 16, 3, 3)`, còn đường tắt là `(N, 8, 6, 6)`. Làm cách nào để cộng được, mà vẫn đúng nghĩa?

A Để broadcasting tự kéo đường tắt cho khớp

B `reshape` đường tắt thành `(N, 16, 3, 3)`

C Bỏ đường tắt ở block này cho khỏi lệch

D Dùng conv $1 \times 1$ stride 2 đưa từ 8 lên 16 kênh

04

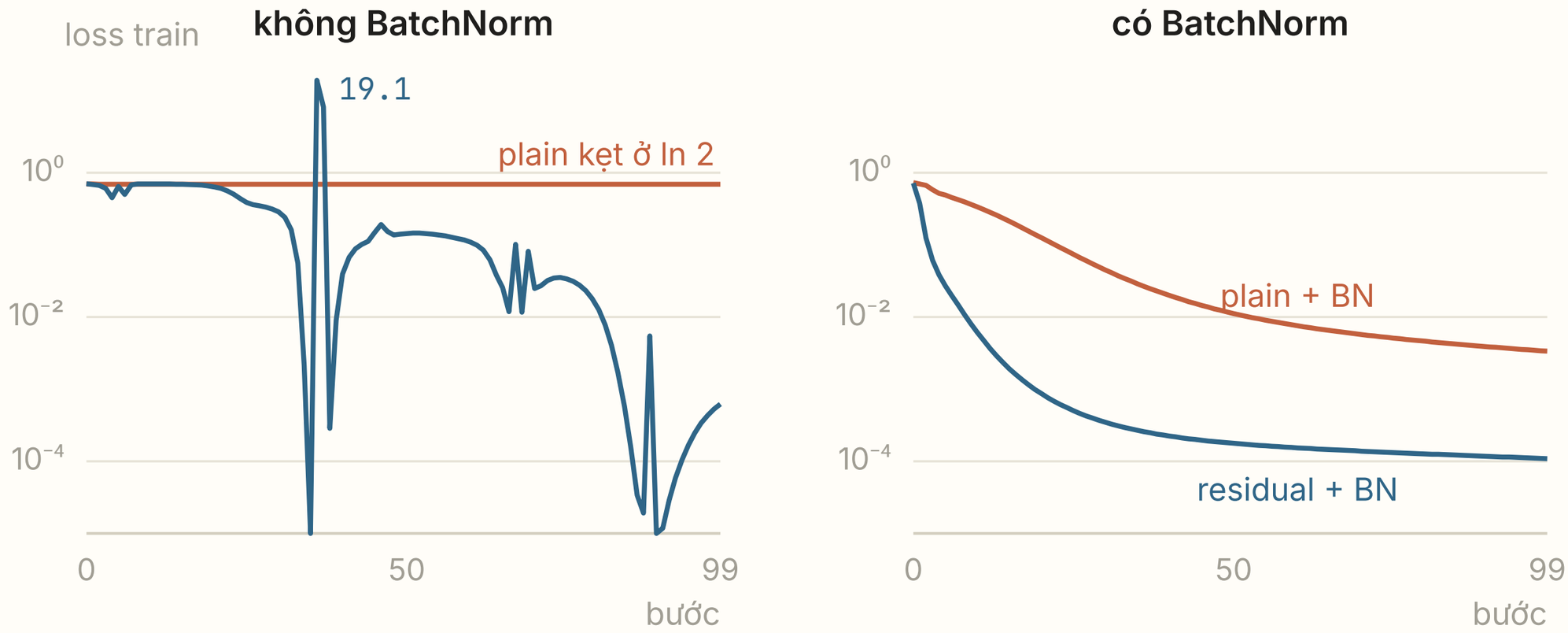
# Mạng 18 lớp có và không có đường tắt

# Bốn tổ hợp, mỗi lần một công tắc

8 block `DeepBlock(skip, norm)` sau conv đầu; cùng seed, Adam, 100 bước.

| | grad | loss@10 | max | loss@99 | valid |
|---|---|---|---|---|---|
| plain | 2.5e-11 | 0.6935 | 0.6980 | 0.6931 | 0.5 |
| plain + BN | 4.1e-01 | 0.3354 | 0.7324 | 0.0034 | 1.0 |
| residual | 3.1e-02 | 0.7010 | 19.1213 | 0.0006 | 1.0 |
| residual + BN | 2.6e-01 | 0.0061 | 0.7157 | 0.0001 | 1.0 |

# Loss theo bước




Mạng 18 lớp: không BatchNorm, mạng thẳng kẹt ở ln 2; residual học được.

# Đọc bảng

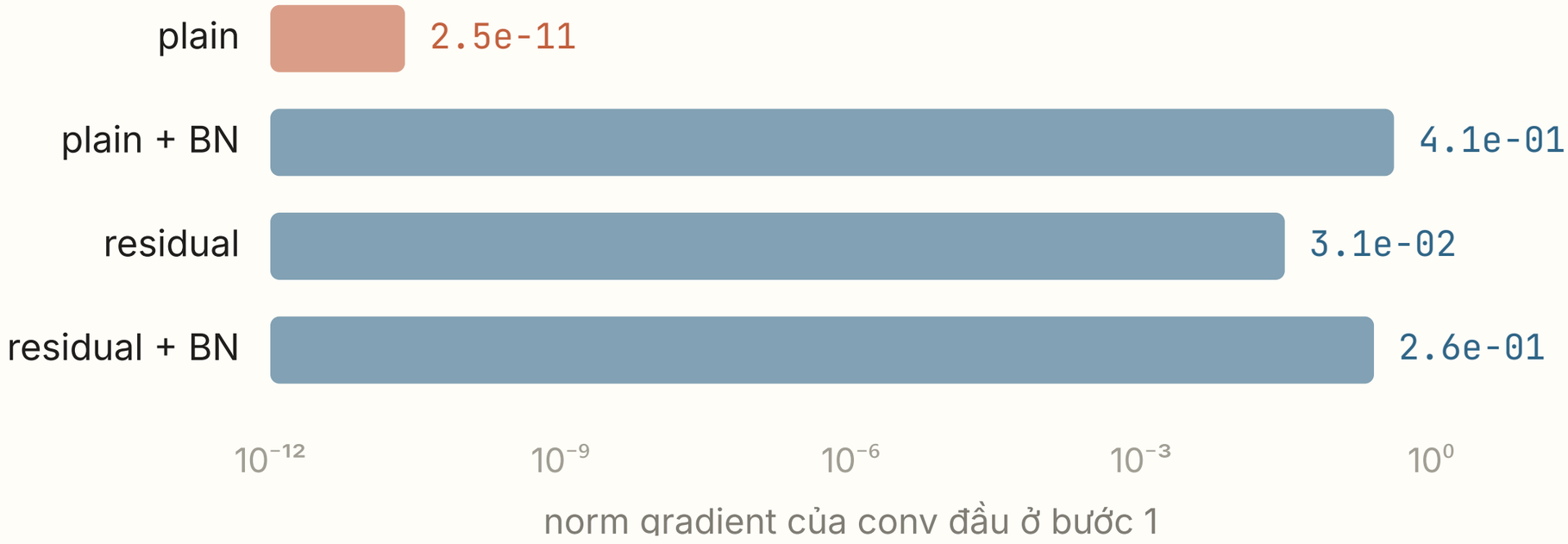
Plain không BN: gradient $2.5 \times 10^{-11}$, kẹt ở $\ln 2$ (như bài 3.3).

Residual không BN: gradient lớn hơn $10^9$ lần nhờ số hạng $I$; học được nhưng có bước loss 19.1213.

Có BN: cả hai học; residual nhanh hơn hẳn (0.0061 so với 0.3354 sau 10 bước).

ResNet = đường tắt + BatchNorm.

# Gradient tới conv đầu




Gradient tới lớp conv đầu: mạng thẳng không BatchNorm gần như bằng 0; có đường tắt hoặc BatchNorm thì không.

HỎI NHANH

# Theo bốn dòng output, thí nghiệm này hỗ trợ kết luận nào?

A Riêng đường tắt đã đủ để train ổn định

B Đã có BatchNorm thì đường tắt trở nên thừa

C Cùng có BatchNorm, mạng có đường tắt hội tụ nhanh hơn

D Mạng thường kẹt ở 0.6931 vì học thuộc tập train

# Ở quy mô ImageNet

Plain 18 lớp và ResNet-18 chính xác gần nhau; ResNet-18 hội tụ nhanh hơn.

Plain 34 lớp có lỗi train cao hơn plain 18 lớp: khó tối ưu, không phải overfit.

05

# ResNet thật

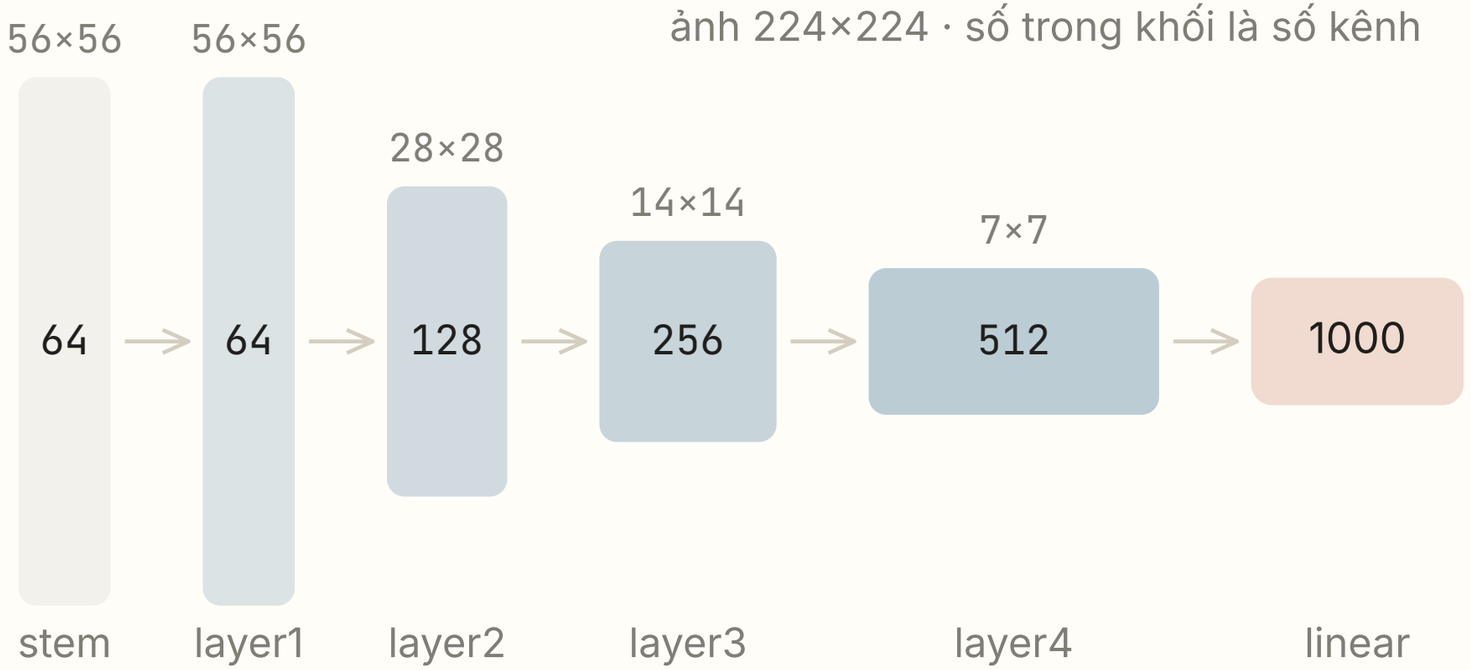
# ResNet-18 qua từng stage

KẾT QUẢ

```
stem   (1, 64, 56, 56)
layer1 (1, 64, 56, 56)
layer2 (1, 128, 28, 28)
layer3 (1, 256, 14, 14)
layer4 (1, 512, 7, 7)
Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
```

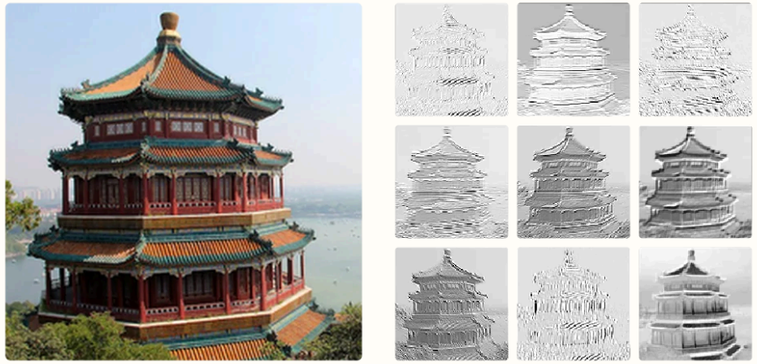
Cạnh giảm 32 lần; block đầu mỗi stage có projection $1 \times 1$ stride 2.

# Cạnh giảm, kênh tăng

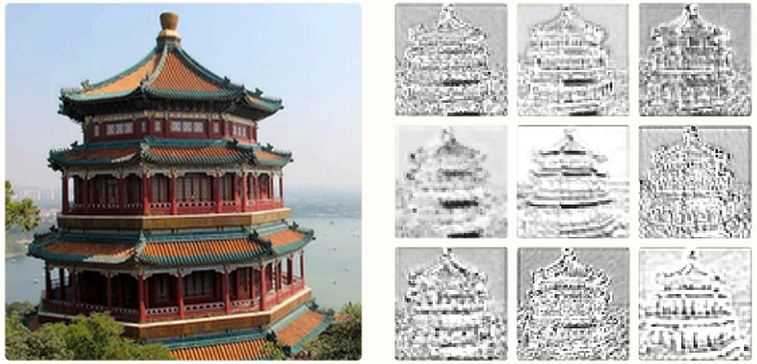



ResNet-18: mỗi stage chia đôi cạnh, nhân đôi số kênh, tới 512 kênh 7×7.

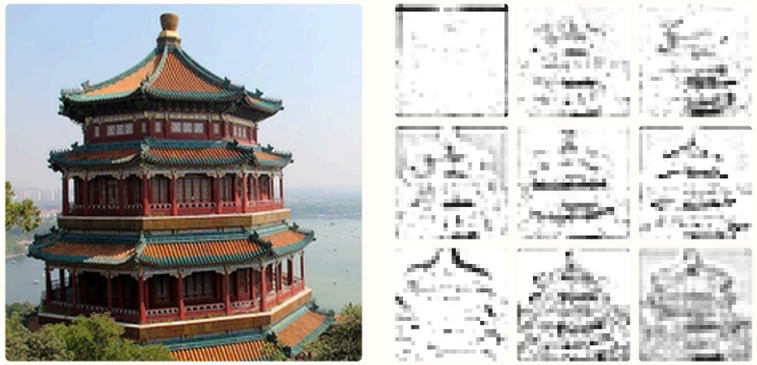
# Feature map thật qua từng stage



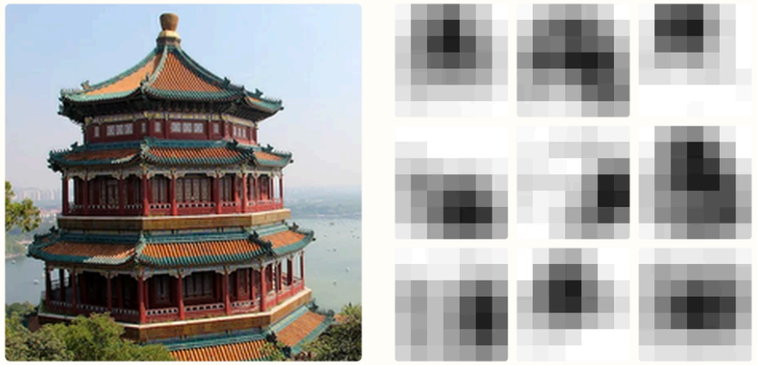
`conv1` · 64 kênh · 112 × 112 (9 kênh mạnh nhất)



`layer1` · 64 kênh · 56 × 56 (9 kênh mạnh nhất)



`layer2` · 128 kênh · 28 × 28 (9 kênh mạnh nhất)



`layer4` · 512 kênh · 7 × 7 (9 kênh mạnh nhất); dự đoán “bell cote” 0.54

Feature map thật của ResNet-18 trên ảnh tháp: càng sâu, bảng càng nhỏ và nét càng trừu tượng.

# Bottleneck

$$256 \cdot 64 + 64 \cdot 64 \cdot 9 + 64 \cdot 256 = 69\,632$$

Hai conv $3 \times 3$ ở 256 kênh: $1\,179\,648$, gấp gần 17 lần.

ResNet-50: khoảng 25.6 triệu tham số, gấp 2.2 lần ResNet-18.

# Bottleneck so với hai conv $3 \times 3$

**bottleneck**

1×1  256 → 64

3×3  64 → 64

1×1  64 → 256

**69 632 tham số**

**hai conv 3×3 ở 256 kênh**

3×3  256 → 256

3×3  256 → 256

**1 179 648 tham số**

ít hơn

**≈ 17 lần**

Bottleneck: conv 1×1 thu kênh lại, conv 3×3 chạy ở ít kênh, conv 1×1 mở ra; ít tham số hơn 17 lần.

HỎI NHANH

`Conv2d(64, 128, 3)` đứng ngay trước `BatchNorm2d(128)`. Để `bias=True` thì thêm bao nhiêu tham số, và chúng có tác dụng gì không?

A 128, nhưng không có tác dụng, vì BatchNorm trừ đi mean của từng kênh

B 128, và có tác dụng giúp BatchNorm hội tụ nhanh hơn

C 8192, nhưng không có tác dụng, vì BatchNorm trừ đi mean của từng kênh

D 8192, và có tác dụng giúp BatchNorm hội tụ nhanh hơn

# Depthwise separable

Depthwise: conv $k \times k$, `groups` bằng số kênh vào, mỗi bộ lọc đọc một kênh.

Pointwise: conv $1 \times 1$ trộn kênh.

$$C_{\text{out}} C_{\text{in}} k^2 \;\to\; C_{\text{in}} k^2 + C_{\text{in}} C_{\text{out}}$$

Nhỏ hơn gần $k^2$ lần.

HỎI NHANH

`nn.Conv2d(64, 64, 3, groups=64, bias=False).weight.shape` là gì?

A `(64, 64, 3, 3)`

B `(64, 1, 3, 3)`

C `(1, 64, 3, 3)`

D `(64, 3, 3)`

# Residual giúp tối ưu, không chống học thuộc

ResNet-50 hơn 25 triệu tham số dễ học thuộc vài trăm ảnh.

Pretrained chỉ dùng khi đề cho phép; luôn so với baseline trên cùng split.

HỎI NHANH

Có 300 ảnh, đề cấm dùng model pretrained. ResNet-50 train từ đầu đạt 100% trên train, 55% trên validation. Một CNN nhỏ đạt 85% trên train, 78% trên validation. Bước tiếp theo hợp lý là gì?

A Giữ mạng nhỏ, thêm augmentation và regularization

B Thêm residual vào ResNet-50 cho dễ tối ưu hơn nữa

C Chọn ResNet-50, vì accuracy trên train cao hơn

D Lên ResNet-101 để học được đặc trưng sâu hơn

# Khi đi thi

- Một batch: shape từng stage, số tham số.
- CNN nhỏ hoặc ResNet-18 trước.
- Pretrained: thay head, đúng preprocessing.

06

# Luyện đề

LUYỆN ĐỀ 1

# Bảng cấu hình ResNet-152 có conv $3 \times 3$, BatchNorm, pooling và phép cộng đường tắt. Thành phần nào là đóng góp riêng của ResNet, giúp train được mạng hơn 100 lớp?

A Đường tắt cộng đầu vào vào đầu ra của mỗi block

B BatchNorm đặt sau mỗi convolution

C Bộ lọc $5 \times 5$ thay cho $3 \times 3$ trong block

D Nhiều lớp pooling hơn giữa các stage

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 216

LUYỆN ĐỀ 2

Một đội train hai CNN thường, không có đường tắt, cùng dữ liệu, cùng BatchNorm và khởi tạo He. Mạng 20 lớp có lỗi 6% trên train, 9% trên validation. Mạng 56 lớp có lỗi 11% trên train, 14% trên validation. Chẩn đoán và cách sửa nào hợp lý?

A Mạng sâu overfit, nên thêm dropout cho mạng 56 lớp

B Gradient quá lớn, nên clip độ lớn gradient về 1

C Mạng sâu khó tối ưu, nên thêm residual connection

D Mạng 56 lớp thừa tham số, nên giảm số kênh

PHỎNG THEO NOAI 2025 Singapore · Phần 1 · Câu 9

LUYỆN ĐỀ 3

Thay một conv $3 \times 3$ thường, từ 64 lên 128 kênh, bằng depthwise separable: depthwise $3 \times 3$ trên từng kênh, tức `groups=64`, rồi pointwise $1 \times 1$ lên 128 kênh. Mọi conv đều không có bias. Số tham số trước và sau là bao nhiêu?

A 73 728 và 1152

B 73 728 và 8768

C 73 728 và 45 056

D 8192 và 8768

PHỎNG THEO NOAI 2025 Singapore · Phần 1 · Câu 14

# Chốt ý

CNN: conv và ReLU tạo feature map, pooling thu nhỏ, head gộp thành nhãn.

Đường tắt $x + F(x)$ giữ đường cho gradient, BatchNorm giữ thang tín hiệu: hai thứ ấy cho phép mạng sâu học được.

# Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook: đổi số block trong thí nghiệm 18 lớp, đoán trước hàng nào kẹt rồi kiểm lại.